

Spesifikasi Rosetta Archive 1.0



Penulis:

Nisrina Zakiyah

18224001

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

1. Ruang lingkup dan istilah

Rosetta Archive (RTA) menyimpan beberapa file dan direktori dalam satu file biner. Format ini memisahkan payload dari indeks. Payload berada setelah header, sedangkan indeks dan footer berada di akhir. Susunan tersebut memungkinkan reader membaca daftar entri dan membuka satu file tanpa memindai payload lain.

Kata HARUS, TIDAK BOLEH, SEBAIKNYA, dan BOLEH menyatakan tingkat kewajiban. Semua offset dihitung dari byte pertama arsip. Rentang byte menggunakan bentuk setengah terbuka `[awal, akhir)`. Penjumlahan offset dan ukuran HARUS diperiksa terhadap overflow sebelum dipakai.

2. Tipe dasar

Nama	Ukuran	Representasi
u8	1	bilangan tanpa tanda
u16	2	little endian
u32	4	little endian
u64	8	little endian
i64	8	two's complement, little endian
bytes[N]	N	byte mentah

CRC32 memakai polinomial IEEE yang digunakan oleh zlib. Nilai awal dan final mengikuti `zlib.crc32`. SHA-256 mengikuti FIPS 180-4. Ketika sebuah CRC struktur dihitung, field CRC milik struktur itu diisi nol terlebih dahulu.

3. Organisasi arsip

```
offset 0
+-----+
| Header, 112 byte |
+-----+ data_offset = 112
| Data file 0 |
| Data file 1 | tiap file dapat store atau zlib
| ... |
+-----+ index_offset
| Index header, 24 byte |
| Entry record 0 | panjang variabel
| Entry record 1 |
```

```

| ... |
+-----+
| Recovery footer, 64 byte |
+-----+ archive_size

```

Direktori tidak memiliki payload. File disusun tanpa padding menurut kenaikan `data_offset`. Indeks HARUS berakhir tepat sebelum footer. Tidak boleh ada byte yang tidak terindeks di data region.

Ukuran arsip memenuhi persamaan berikut:

```
archive_size = 112 + jumlah(stored_size file) + index_size + 64
```

4. Header arsip

Header berukuran 112 byte.

Offset	Ukuran	Field	Aturan versi 1.0
0	8	magic	52 54 41 1A 0D 0A 00 00
8	2	major	1
10	2	minor	0
12	4	header_size	112
16	4	flags	bitmask yang dijelaskan di bawah
20	4	entry_count	jumlah record indeks
24	8	index_offset	awal indeks
32	8	index_size	ukuran indeks lengkap
40	8	data_offset	112
48	8	archive_size	sama dengan ukuran file
56	8	created_ns	0 untuk keluaran deterministik
64	32	index_sha256	SHA-256 indeks lengkap
96	4	header_crc32	CRC32 seluruh header dengan field ini nol
100	12	reserved	seluruh byte nol

Flag arsip:

Bit	Nama	Makna
0	DETERMINISTIC	field pembuatan tidak memakai waktu saat ini
1	PER_ENTRY_COMPRESSION	method tersimpan pada setiap record
2	RECOVERY_FOOTER	footer 64 byte tersedia

Writer versi 1.0 HARUS mengaktifkan ketiga bit. Reader versi 1.0 HARUS menolak bit lain karena maknanya belum diketahui.

5. Data region

Data region dimulai pada `data_offset` dan berakhir pada `index_offset`. Hanya entri file yang mendapat rentang data. Record indeks menyimpan offset absolut dan ukuran payload tersimpan.

Method 0 bernama `store`. Payload identik dengan isi file dan `stored_size` HARUS sama dengan `original_size`.

Method 1 bernama `zlib`. Payload adalah satu zlib stream lengkap seperti yang didefinisikan RFC 1950. Hasil dekompresi HARUS tepat sebesar `original_size`. Decoder HARUS menolak stream truncated, output yang melewati ukurandeklarasi, atau byte tambahan setelah akhir stream.

File kosong memakai method `store`, ukuran nol, dan `data_offset` yang sama dengan cursor data saat itu. Rentang kosong sah dan tidak mengonsumsi byte.

6. Index header

Indeks dimulai dengan header 24 byte.

Offset relatif	Ukuran	Field	Aturan
0	8	<code>magic</code>	ASCII <code>RTAIDX</code> diikuti dua byte nol
8	2	<code>major</code>	1
10	2	<code>minor</code>	0
12	4	<code>entry_count</code>	sama dengan header arsip
16	8	<code>total_size</code>	sama dengan <code>index_size</code>

Record mengikuti header tanpa padding. Jumlah record HARUS sama dengan `entry_count`, dan record terakhir HARUS berakhir tepat pada `total_size`.

7. Entry record

Setiap record terdiri dari fixed part 108 byte, path UTF-8, lalu extension area. `record_size` mencakup ketiganya.

Offset relatif	Ukuran	Field	Aturan
0	4	magic	ASCII <code>REN1</code>
4	4	record_size	minimal <code>108 + path_length</code>
8	1	entry_type	1 file, 2 directory
9	1	entry_flags	0 pada versi 1.0
10	1	method	0 store, 1 zlib
11	1	reserved8	0
12	4	path_length	panjang path dalam byte
16	4	mode	bit permission POSIX, hanya <code>0o7777</code>
20	4	uid	uid numerik asal
24	4	gid	gid numerik asal
28	8	mtime_ns	Unix time dalam nanosecond
36	8	original_size	ukuran isi setelah dekompresi
44	8	stored_size	ukuran payload di arsip
52	8	data_offset	offset absolut payload
60	4	data_crc32	CRC32 isi asli
64	4	stored_crc32	CRC32 payload tersimpan
68	32	sha256	SHA-256 isi asli
100	4	record_crc32	CRC32 fixed part, path, dan extension
104	4	reserved32	seluruh byte nol
108	path_length	path	UTF-8 canonical seperti bagian 8
berikutnya	variabel	extensions	<code>record_size - 108 - path_length</code> byte

Untuk direktori, `method`, kedua ukuran, `data_offset`, dan kedua CRC HARUS nol. SHA-256 HARUS sama dengan hash string kosong:

```
e3b0c44298fc1c149afb4c8996fb924
27ae41e4649b934ca495991b7852b855
```

Record file dengan method `store` HARUS memiliki `stored_size == original_size`.

8. Aturan path dan organisasi entri

Path memakai UTF-8 strict dan Unicode Normalization Form C. Separator selalu `/`. Path TIDAK BOLEH:

1. kosong
2. diawali `/`
3. mengandung NUL
4. memiliki komponen kosong, `..`, atau `..`
5. memakai prefix drive seperti `c:`
6. lebih panjang dari 1 MiB setelah encoding UTF-8

Record diurutkan menaik menurut byte UTF-8 path. Setiap path HARUS unik. Jika sebuah path memiliki parent selain `..`, parent tersebut HARUS memiliki record yang lebih dulu atau lebih belakangan di indeks dengan tipe `directory`. Aturan parent diperiksa setelah seluruh indeks selesai dibaca.

Writer menyimpan nama input sebagai root arsip. Sebagai contoh, input `data/foto` menjadi root `foto`, bukan path absolut host.

9. Metadata

RTA 1.0 menyimpan metadata berikut:

Metadata	Alasan
tipe entri	membedakan file dari direktori, termasuk direktori kosong
mode	mengembalikan permission dan bit eksekusi
uid dan gid	mencatat kepemilikan asal untuk inspeksi atau restore administratif
mtime nanosecond	mempertahankan waktu modifikasi dengan presisi filesystem
ukuran asli	membatasi dekompresi dan mendeteksi truncation
method dan ukuran tersimpan	menemukan serta membaca payload secara mandiri

CRC32 dan SHA-256	deteksi kerusakan cepat dan pemeriksaan isi yang kuat
-------------------	---

Nama platform, access time, birth time, ACL, extended attribute, hard link, dan symlink tidak termasuk versi 1.0. Reader tidak boleh mengarang metadata tersebut.

10. Extension area

Extension area adalah urutan TLV tanpa padding. Header TLV berukuran 8 byte:

Offset	Ukuran	Field
0	2	extension_type, tidak boleh nol atau duplikat
2	2	flags, bit 0 berarti critical
4	4	payload_size
8	variabel	payload

Versi 1.0 belum menetapkan extension type. Reader HARUS melewati extension yang tidak dikenal jika flag critical tidak aktif. Reader HARUS menolak extension critical yang tidak dikenal, flag yang tidak dikenal, TLV truncated, atau panjang yang melewati record. CRC record melindungi seluruh extension area.

11. Recovery footer

Footer 64 byte berada tepat di akhir arsip.

Offset relatif	Ukuran	Field	Aturan
0	8	magic	ASCII RTAEND diikuti dua byte nol
8	2	major	1
10	2	minor	0
12	4	flags	sama dengan header
16	8	index_offset	sama dengan header
24	8	index_size	sama dengan header
32	8	archive_size	sama dengan ukuran file
40	8	header_offset	0
48	4	index_crc32	CRC32 indeks lengkap
52	4	footer_crc32	CRC32 footer dengan field ini nol

56	8	reserved	seluruh byte nol
----	---	----------	------------------

Duplikasi lokasi indeks bersifat sengaja. Tool pemulihan dapat menemukan indeks dari akhir file ketika header rusak. Reader normal HARUS meminta header dan footer konsisten.

12. Algoritma pembuatan canonical

Writer canonical menjalankan langkah berikut:

1. Tolak symlink dan filesystem object selain file biasa atau direktori.
2. Bentuk path relatif, normalisasi NFC, lalu deteksi collision.
3. Sertakan record untuk setiap direktori, termasuk yang kosong.
4. Urutkan entri menurut byte UTF-8 path.
5. Baca file, hitung CRC32 serta SHA-256 isi asli.
6. Kompres dengan zlib. Pakai hasilnya hanya jika lebih kecil, kecuali pengguna memilih `--store`.
7. Letakkan payload file berurutan mulai offset 112.
8. Bangun record indeks, index header, dan hash indeks.
9. Bangun header serta footer dengan CRC masing-masing.
10. Tulis ke file sementara, `fsync`, lalu rename atomik ke target.

13. Algoritma parsing dan verifikasi

Reader normal HARUS melakukan urutan pemeriksaan berikut sebelum ekstraksi:

1. Pastikan ukuran minimum cukup untuk header, index header, dan footer.
2. Baca header fixed size. Periksa magic, versi, flag, reserved, CRC, dan ukuran file.
3. Validasi `data_offset`, `index_offset`, `index_size`, dan posisi footer dengan aritmetika aman.
4. Baca footer. Periksa CRC dan kesesuaiannya dengan header.
5. Batasi `entry_count`, `index_size`, serta `path_length` sebelum alokasi besar.
6. Baca indeks. Periksa SHA-256 dari header dan CRC32 dari footer.

7. Parse record satu per satu. Periksa ukuran, CRC, UTF-8, canonical path, extension, tipe, method, dan metadata.
8. Periksa sorting, duplikat, parent directory, rentang data, gap, overlap, dan konsistensi ukuran.
9. Untuk verifikasi penuh, baca setiap payload. Periksa `stored_crc32`, decoder zlib, ukuran hasil, `data_crc32`, dan SHA-256.

`list`, `info`, dan `inspect` menyelesaikan langkah struktur 1 sampai 8 tanpa mengekstrak payload. `verify` menyelesaikan semua langkah. `extract` memanggil verifikasi penuh sebelum menulis output.

Reader HARUS gagal dengan pesan kesalahan dan tidak mencoba menebak nilai ketika menemukan struktur yang ambigu. Arsip corrupt boleh ditolak.

14. Ekstraksi aman

Selain validasi path di indeks, extractor HARUS:

1. menggabungkan path hanya di bawah output root
2. menolak output root berupa symlink
3. memeriksa setiap komponen yang sudah ada dan menolak symlink
4. menolak overwrite file atau direktori yang konflik
5. memakai operasi create exclusive untuk file
6. memakai `O_NOFOLLOW` jika platform menyediakannya
7. menerapkan metadata setelah isi file selesai ditulis
8. menerapkan metadata direktori dari yang paling dalam agar mtime parent tidak berubah lagi

Penerapan uid dan gid BOLEH menjadi opsi eksplisit karena biasanya memerlukan hak administratif. Nilainya tetap HARUS tersedia untuk inspeksi.

15. Versioning dan compatibility

Versi terdiri dari `major.minor` pada header, index header, dan footer.

1. Perubahan major boleh mengubah layout atau arti field. Reader 1.x HARUS menolak major selain 1.

2. Perubahan minor mempertahankan layout dasar, tetapi dapat menambah aturan atau fitur. Reader HARUS menolak minor yang lebih baru jika tidak dapat menjamin semantiknya.
 3. Extension record yang optional dapat dibaca lintas implementasi tanpa menaikkan major.
 4. Extension critical memaksa reader lama berhenti sehingga data tidak ditafsirkan dengan aturan yang salah.
 5. Reserved field tetap nol sampai revisi format secara resmi memberi arti.
- Writer 1.0 hanya menghasilkan versi 1.0. Implementasi referensi menolak revisi yang tidak didukung sebelum membaca payload.

16. Determinisme

Writer canonical menetapkan `created_ns = 0`, tidak menyimpan UUID acak, mengurutkan path berdasarkan byte UTF-8, memakai parameter kompresi tetap, dan tidak menambahkan padding acak. Dua proses dengan isi file, nama, metadata, opsi, implementasi, dan versi zlib yang sama HARUS menghasilkan byte arsip yang sama.

`mtime`, `mode`, `uid`, dan `gid` adalah bagian input. Perubahan metadata tersebut berarti input tidak lagi identik. Output zlib dijamin deterministik untuk library dan parameter yang sama. Spesifikasi tidak menjamin stream terkompresi yang samadari dua implementasi zlib berbeda, tetapi isi hasil dekompresi dan seluruh metadata harus sama.

17. Random access dan streaming

Reader dapat menemukan indeks dari header atau footer, lalu melakukan seek langsung ke `data_offset` file yang dipilih. Verifikasi dan ekstraksi memproses payload dalam chunk 1 MiB sehingga tidak memerlukan buffer sebesar isi hasildekompresi.

Pembuatan oleh implementasi referensi menahan isi input di memori agar offset dan indeks dapat dihitung sebelum atomic write. Ini batas implementasi, bukan batas format. Writer lain boleh melakukan dua pass atau memakai file sementara untuk input besar.

18. Pemulihan parsial

Perintah `recover` membaca footer dari akhir file, mengambil region indeks, lalu mencari magic record `REN1`. Setiap calon record harus lulus CRC, path, extension, batas payload, CRC payload, dekompresi, ukuran asli, CRC data, dan SHA-256. Record rusak dilewati. Parent directory yang hilang dibuat kembali dengan metadata aman default. Hasil selalu ditulis sebagai arsip baru dan tidak mengubah sumber.

Pemulihan tidak dijamin jika footer rusak, index region tidak dapat ditemukan, atau tidak ada satu pun record dan payload yang valid. Pencarian magic hanya membantu menemukan batas calon record. Semua checksum tetap wajib, sehinggabyte acak tidak langsung dipercaya sebagai entri.

19. Batas implementasi referensi

1. File biasa dan direktori didukung. Symlink serta special file ditolak.
2. Batas parser: 1.000.000 entri, indeks 256 MiB, path 1 MiB.
3. Tidak ada enkripsi, autentikasi pengirim, ACL, xattr, hard link, atau sparse map.
4. SHA-256 mendeteksi perubahan isi, tetapi tanpa tanda tangan digital penyerang yang dapat menulis ulang seluruh arsip juga dapat menghitung checksum baru.
5. Archive append in place belum didukung. Pembuatan selalu menghasilkan satu snapshot baru.

20. Conformance minimum

Sebuah writer conforming HARUS menghasilkan semua struktur dan invariant pada bagian 3 sampai 12. Sebuah reader conforming HARUS melakukan pemeriksaan bagian 13 serta aturan keamanan bagian 14 sebelum ekstraksi. Implementasi yang hanya menampilkan listing tetap tidak boleh mengabaikan batas indeks, CRC struktur, path, duplikat, dan version compatibility.